PASO A: DEMOSTRACIÓN PRÁCTICA

Análisis del Código Vulnerable y Explotación de Comandos RCE

Demostración de inyección de comandos paso a paso: desde el script PHP inseguro hasta la extracción del archivo `/etc/passwd` en Xubuntu.

Entorno: **Xubuntu + XAMPP / Apache**

Fallo: **Inyección Directa en shell_exec()**

MALA PRÁCTICA DE DESARROLLO

El Script Inseguro: ping.php

```php
// Código vulnerable de ejemplo if (isset($_GET['ip'])) { $ip =
$_GET['ip']; // VULNERABILIDAD RCE: Concatenación directa sin filtrar
$resultado = shell_exec("ping -c 4 " . $ip); echo "$resultado"; } ?>
```

⚠ Falla Principal

Confianza ciega en la variable `$_GET['ip']`. La aplicación une texto proporcionado por el usuario directamente con un comando de sistema operativo.

⚙ Intención Original

Ofrecer una herramienta diagnóstica rápida para validar conectividad con servidores o impresoras desde una interfaz web.

MECANISMO DE EXPLOTACIÓN

Paso a Paso: Del Uso Normal al Ataque

1. SOLICITUD LEGÍTIMA

```
http://localhost/ping.php?ip=8.8.8.8
```

Comando ejecutado en Linux:

```
ping -c 4 8.8.8.8
```

→

2. INYECCIÓN DE COMANDOS (RCE)

```
http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd
```

El operador `;` divide e introduce un 2º comando:

```
ping -c 4 8.8.8.8 ; cat /etc/passwd
```

RESULTADO DE LA EXPLOTACIÓN

Respuesta del Servidor Xubuntu / XAMPP

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data. 64 bytes from 8.8.8.8: icmp_seq=1
ttl=117 time=14.2 ms --- 8.8.8.8 ping statistics --- 4 packets transmitted, 4
received, 0% packet loss root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin www-data:x:33:33:www-
data:/var/www:/usr/sbin/nologin mysql:x:997:1001::/home/mysql:/bin/sh
aristides1000:x:1000:1000:aristides1000:/home/aristides1000:/bin/bash
```

Comando Ping Ejecutado

Procesa la primera orden requerida por la aplicación web.

Inyección Exitosa

Imprime inmediatamente las líneas solicitadas del archivo del sistema.

PRUEBAS ADICIONALES DE DIAGNÓSTICO

Otros Comandos Comunes en Pruebas RCE

`; whoami`

Salida: daemon / www-data

Identifica el usuario del proceso web para medir el alcance de los permisos iniciales.

`; id`

Salida: uid=1@ (daemon) gid=1@ ...

Muestra grupos e identificadores numéricos del proceso actual.

`; uname -a`

Salida: Linux xubuntu 6.8.0 ...

Revela versión del Kernel y arquitectura del sistema operativo.

`; ls -la`

Salida: drwxr-xr-x ping.php ...

Lista los archivos y permisos del directorio web actual.

ANÁLISIS DE IMPACTO

¿Por qué este resultado representa un riesgo crítico?

Identificación de Usuarios

Revela la existencia de cuentas reales con acceso interactivo (`aristides1000`, `root`) ideales para fuerza bruta SSH.

Reconoce Servicios

Confirma la presencia de software activo en la máquina objetivo mediante usuarios de sistema como `mysql`.

Prueba de Concepto (PoC)

Es la evidencia irrefutable de que el servidor web aceptará **cualquier comando** arbitrario enviado por el atacante.

CONCLUSIÓN PARA LA AUDIENCIA

De una Casilla Diagnóstica al Control Absoluto del Servidor

El riesgo no es solo leer archivos del sistema. Si el servidor procesa `cat /etc/passwd`, un atacante también puede borrar directorios, descargar malware o tomar el control mediante consolas remotas.

Siguiente Paso: Auditoría del código usando Prompts de IA (Paso B)

PASO B: TALLER PRÁCTICO (10 MIN)

Auditoría de Código Seguro con DeepSeek / IA

Uso de la Inteligencia Artificial conversacional como auditor senior de ciberseguridad para detección acelerada de vulnerabilidades RCE.

Herramienta: **DeepSeek / ChatGPT / Claude**

Metodología: **Prompt Engineering Estructurado**

PROMPT PARA ASISTENTES

Prompt Especializado de Ciberseguridad

COPIA Y PEGA ESTE TEXTO EN DEEPSEEK

Actúa como un Auditor de Ciberseguridad Senior y Analista de Código Seguro en PHP.
Revisa el siguiente fragmento de código en busca de vulnerabilidades críticas de ejecución remota de comandos (RCE):

```
$resultado"; } ?>
```

Entrégame tu análisis estructurado de la siguiente manera:
1. Identificación exacta de la línea vulnerable y el riesgo asociado (PHP2Shell / RCE).
2. Explicación de cómo un atacante podría explotarlo.
3. El código corregido y seguro aplicando las mejores prácticas de saneamiento y funciones seguras (ej. escapes o listas blancas).

ANÁLISIS DE LA RESPUESTA DE LA IA

¿Qué debe responder DeepSeek en Vivo?

01. Detección

Identificación Precisa

Señala la concatenación directa dentro de `shell_exec()` como un vector RCE crítico.

02. Vector

Mecanismo de Ataque

Explica el uso de operadores de consola (`;`, `&&`, `|`) para inyectar comandos no autorizados.

03. Parche

Código Sanitizado

Entrega código corregido usando `filter_var()` y `escapeshellarg()`.

PASO C: ANÁLISIS DE REMEDIACIÓN (5 MIN)

Análisis del Código Seguro propuesto por la IA

Comprensión de los mecanismos defensivos: Escapado de argumentos de shell y validación estricta con filtros nativos de PHP.

Sanitización: **escapeshellarg()**

Validación: **filter_var()**

REMEDIACIÓN RECOMENDADA

El Script PHP Corregido y Seguro

```php
if (isset($_GET['ip'])) { $ip = $_GET['ip']; // 1. VALIDACIÓN ESTRICTA: ¿Es una IP legítima? if (filter_var($ip, FILTER_VALIDATE_IP)) { // 2. SANITIZACIÓN: Escapar caracteres especiales de consola $safe_ip = escapeshellarg($ip); $resultado = shell_exec("ping -c 4 " . $safe_ip); echo "$resultado"; } else { echo "Error: Formato de dirección IP no válido."; } } ?>
```

Doble Capa de Defensa

No confía en la entrada. Combina verificación de formato con neutralización de metacaracteres.

Resultado de Seguridad

Bloquea cualquier intento de inyección de comandos como `; cat /etc/passwd` antes de llegar a la terminal.

MECANISMOS DE DEFENSA

¿Cómo funcionan las soluciones sugeridas?

MECANISMO 1

escapeshellarg()

Envuelve la cadena entre comillas simples y escapa caracteres de control de la consola.

```
Entrada: 8.8.8.8; cat /etc/passwd
Salida: '8.8.8.8; cat /etc/passwd'
```

MECANISMO 2 (RECOMENDADO)

filter_var(..., FILTER_VALIDATE_IP)

Valida que el string recibido coincida estrictamente con la estructura de una dirección IPv4 o IPv6.

```
8.8.8.8 → VÁLIDO (TRUE)
8.8.8.8; cat /etc/passwd → INVÁLIDO (FALSE)
```

LECCIÓN DE DESARROLLO SEGURO

Nunca confíes en los datos recibidos del exterior

La IA nos ayuda a reducir drásticamente el tiempo de remediación, permitiendo pasar de un código vulnerable a un parche robusto en cuestión de segundos.

A continuación: Bloque 4 - Mitigación Urgente y Hardening de Servidores (15 min)

BLOQUE 4 / PASO D: MITIGACIÓN Y HARDENING (15 MIN)

Hardening de Servidores y Defensa en Profundidad

Estrategias de contención inmediata, permisos de sistema operativo, principios de desarrollo seguro y monitoreo con WAF y Logs.

Infraestructura: **php.ini + Permisos Linux**

Monitoreo: **WAF + Logs de Servidor**

CASO PRÁCTICO 1: PARCHEO EN PHP.INI

Desactivación Global de Funciones Peligrosas

```
; Archivo de configuración php.ini disable_functions = exec, passthru, shell_exec, system, proc_open, popen
```

Resultado al ejecutar el exploit:

```
Warning: shell_exec() has been disabled for security reasons in /opt/lampp/htdocs/ping.php
```

Sellado de Puerta Trasera

Incluso si existe un fallo en el código PHP, el motor se niega rotundamente a invocar binarios del sistema operativo.

CASO PRÁCTICO 2: PERMISOS EN LINUX

Principio de Menor Privilegio (Xubuntu)

MALA CONFIGURACIÓN (777)

```
sudo chmod -R 777 /opt/lampp/htdocs/
```

El atacante puede crear o sobrescribir archivos del sitio web mediante la inyección de comandos.

HARDENING APLICADO (644 / 755)

```
sudo chown -R www-data:www-data /opt/lampp/htdocs/
sudo chmod 755 /opt/lampp/htdocs/
```

Bloquea la creación de archivos. La consola devuelve: `Permission denied`.

CASO PRÁCTICO 3: DESARROLLO SEGURO

Cero Confianza en la Entrada Externa

Prohibir eval()

Eliminar cualquier evaluación dinámica de texto como código fuente.

Filtrar GET/POST/Cookies

Validar tipos de datos y estructuras antes de procesar la lógica de negocio.

Listas Blancas

Aceptar únicamente valores conocidos y previamente permitidos.

CASO PRÁCTICO 4: DETECCIÓN EN TIEMPO REAL

Monitoreo de Logs y WAF

Monitoreo de terminal en tiempo real (tail -f access.log):

```
127.0.0.1 - - [29/Oct/2026:21:00:00 -0400] "GET /ping.php?
ip=8.8.8.8;cat%20/etc/passwd HTTP/1.1" 200 1450
```

Análisis Automatizado

Detección de patrones de escape (`;`, `&&`, `|`) en peticiones HTTP.

Acción del WAF

Bloquea la conexión y la dirección IP sospechosa antes de tocar la aplicación.

RESUMEN FINAL DE LA MASTERCLASS

Estrategia Integral: Código Seguro + Servidor Protegido

La ciberseguridad requiere múltiples capas: validación estricta en desarrollo, auditoría ágil con IA y hardening robusto a nivel de infraestructura.

¡Gracias por su atención! Espacio abierto para Preguntas y Respuestas (Q&A)